

1 Práctica Final de Sistemas Distribuidos

1.1 1. Portada

Título: Servicio distribuido de envío de mensajes

Asignatura: Sistemas Distribuidos

Curso: 2025-2026

Autores:

Jose Luis Palacios | 100495679 |

Javier Olozaga | 100522178 |

1.2 2. Índice

1. Portada
2. Índice
3. Introducción y objetivo de la práctica
4. Arquitectura general del sistema
5. Descripción del cliente
6. Descripción del servidor
7. Servicio web
8. Servicio RPC
9. Compilación y ejecución
10. Pruebas realizadas
11. Problemas encontrados y soluciones
12. Conclusiones

1.3 3. Introducción y objetivo de la práctica

El objetivo de la práctica es implementar una aplicación distribuida de mensajería entre usuarios. El sistema permite registrar usuarios, conectarlos y desconectarlos del servicio, consultar los usuarios conectados, enviar mensajes de texto y entregar mensajes que se hubieran quedado pendientes cuando el destinatario estaba desconectado.

La práctica combina varias tecnologías de comunicación distribuidas. La funcionalidad principal se implementa mediante sockets TCP entre clientes Python y un servidor de mensajería en C. La ampliación incorpora transferencia de ficheros entre clientes, un servicio web en Python para normalizar mensajes y un servicio ONC-RPC en C para registrar las operaciones que recibe el servidor principal.

El sistema implementado mantiene la separación de responsabilidades indicada en el enunciado. El servidor de mensajería conserva el estado global de usuarios y mensajes pendientes. Los clientes se encargan de interactuar con el usuario, escuchar entregas entrantes y transferir ficheros directamente entre ellos. El servicio web se despliega de forma local en la máquina donde se ejecuta cada cliente. El servicio RPC actúa como un componente auxiliar de registro, invocado por el servidor de mensajería cuando recibe operaciones.

La implementación se ha desarrollado buscando preservar la compatibilidad de la primera parte al introducir las ampliaciones. Por ello, las operaciones SENDATTACH, GETFILE, servicio web y RPC se han añadido sin modificar el protocolo básico de REGISTER, UNREGISTER, CONNECT, DISCONNECT, USERS y SEND, salvo la ampliación obligatoria de USERS para devolver también IP y puerto.

1.4 4. Arquitectura general del sistema

La arquitectura está formada por cuatro componentes principales: cliente Python, servidor de mensajería en C, servicio web de normalización y servidor RPC de registro de operaciones.

El cliente Python (`client.py`) proporciona la interfaz de comandos. Desde esta interfaz se ejecutan las operaciones de usuario: registro, baja, conexión, desconexión, consulta de usuarios, envío de mensajes, envío de mensajes con nombre de fichero adjunto y solicitud de transferencia de fichero. El cliente es concurrente: además del hilo que atiende la consola, crea un hilo de escucha cuando un usuario se conecta. Este hilo recibe mensajes procedentes del servidor y peticiones directas de otros clientes.

El servidor de mensajería (`server.c`) es un servidor TCP concurrente. Escucha en el puerto indicado por línea de comandos y crea un hilo por cada conexión aceptada. Cada conexión corresponde a una operación del protocolo. El servidor mantiene en memoria la lista de usuarios registrados, su estado de conexión, la dirección IP y puerto de escucha cuando están conectados, el último identificador de mensaje asignado a cada usuario y la lista de mensajes pendientes de entrega.

El servicio web (`web_service.py`) es un servidor HTTP local. Su función es recibir un mensaje de texto, eliminar espacios en blanco repetidos y devolver el mensaje normalizado. El cliente llama a este servicio antes de enviar SEND o SENDATTACH. De esta forma, los mensajes almacenados y entregados por el servidor ya se encuentran normalizados.

El servicio RPC se define mediante `log_rpc.x` y se implementa con ficheros generados por `rpcgen` más una función de servicio en `log_rpc_server.c`. El servidor de mensajería actúa como cliente RPC utilizando

`log_rpc_client.c`. Cada vez que procesa una operación de usuario, envía al servidor RPC el usuario, la operación y, en el caso de `SENDATTACH`, el nombre del fichero.

La comunicación principal utiliza sockets TCP. Cliente y servidor usan cadenas terminadas en `\0` para los campos textuales, y bytes binarios para los códigos de resultado. Para la transferencia de ficheros entre clientes se utiliza también TCP, pero el contenido del fichero se envía como bytes binarios precedidos por el tamaño total.

1.5 5. Descripción del cliente

El cliente se implementa en `client.py` y mantiene el formato de interfaz proporcionado inicialmente. El programa se ejecuta indicando la IP y el puerto del servidor principal. Internamente guarda estos datos en atributos de clase y utiliza conexiones TCP nuevas para cada operación enviada al servidor.

La operación `REGISTER` abre una conexión con el servidor, envía la cadena de operación y el nombre del usuario, recibe un byte de resultado y muestra el mensaje correspondiente. `UNREGISTER` funciona de forma análoga, eliminando además el estado local si el usuario dado de baja era el conectado en ese proceso cliente.

La operación `CONNECT` es la que activa la concurrencia del cliente. Antes de enviar la solicitud al servidor, el cliente crea un socket de escucha en un puerto libre y arranca un hilo que acepta conexiones entrantes. Después envía al servidor el usuario y el puerto elegido. Si la conexión se acepta, el cliente queda asociado a ese usuario. Si falla, se cierra el socket de escucha para no dejar recursos abiertos. `DISCONNECT` solicita la desconexión al servidor y, tanto si la operación termina correctamente como si se produce un error, detiene el hilo de escucha local cuando corresponde.

La operación `USERS` solicita al servidor la lista de usuarios conectados. En la versión final, cada entrada recibida tiene el formato `usuario :: IP :: puerto`. El cliente imprime estas entradas y, además, las guarda en el diccionario interno `_known_users`. Esta estructura permite localizar posteriormente a otro cliente para realizar transferencias de ficheros directas mediante `GETFILE`.

La operación `SEND` envía un mensaje de texto al servidor. Antes de enviarlo, el cliente invoca el servicio web de normalización. Si el servicio web responde correctamente, se comprueba que el mensaje no supere el límite de 255 bytes útiles y se envía al servidor junto con el usuario remitente y el destinatario. El servidor devuelve un identificador numérico si la operación se almacena correctamente.

La operación `SENDATTACH` amplía `SEND` incorporando el nombre de un fichero. El contenido del fichero no viaja por el servidor de mensajería. El servidor solo almacena y entrega el texto del mensaje, el identificador y el nombre del fichero. El comando implementado es `SENDATTACH <userName> <message> <fileName>`, tomando el último argumento como ruta del fichero y el texto intermedio como mensaje.

La operación `GETFILE` permite recuperar el contenido de un fichero desde otro cliente. Para ello se consulta `_known_users`; si no existe información del usuario remoto, el cliente ejecuta internamente `USERS` para refrescar la caché. Si el usuario sigue sin aparecer conectado, se informa de que la transferencia no puede realizarse. Si se conoce su IP y puerto, se abre una conexión TCP al hilo de escucha del otro cliente y se envía la operación `GET FILE`, el usuario solicitante y el nombre del fichero remoto. El cliente remoto responde con `OK` o `ERROR`; si responde `OK`, envía el tamaño y después los bytes del fichero.

El hilo de escucha del cliente procesa varias operaciones entrantes. Para mensajes normales recibe `SEND MESSAGE` y muestra el remitente, identificador y texto. Para mensajes con adjunto recibe `SEND MESSAGE ATTACH`, muestra el mensaje y añade una línea `FILE <fileName>`. También procesa las confirmaciones `SEND MESS ACK` y `SEND MESS ATTACH ACK`. Finalmente, atiende `GET FILE` leyendo el fichero local solicitado y enviando su contenido si existe y puede abrirse.

1.6 6. Descripción del servidor

El servidor principal se implementa en C en `server.c` y se compila como ejecutable `server`. Se ejecuta con `./server -p <port>`. Al arrancar crea un socket TCP, hace `bind`, `listen` y entra en un bucle de aceptación de conexiones. Por cada conexión aceptada reserva una estructura con el descriptor y la IP del cliente, crea un hilo y delega en este la lectura y ejecución de la operación.

El estado global del servidor se define mediante una lista enlazada de usuarios. Cada usuario contiene nombre, estado (`USER_CONNECTED` o `USER_DISCONNECTED`), IP, puerto de escucha, último identificador de mensaje asignado y una cola de mensajes pendientes. La cola se implementa con nodos `pending_message_t`, que almacenan identificador, remitente, destinatario, texto, indicador de adjunto y nombre del fichero si procede.

La concurrencia se controla con un mutex global (`users_mutex`) que protege la lista de usuarios y las colas de pendientes. Las funciones que consultan o modifican usuarios se ejecutan bajo este bloqueo. Para evitar bloquear todo el servidor durante operaciones de red, cuando se va a enviar un mensaje a un cliente se copian los datos necesarios, se libera el mutex, se realiza la comunicación y después se vuelve a tomar el mutex solo para actualizar el estado o eliminar el mensaje pendiente.

`REGISTER` comprueba si el usuario existe. Si no existe, crea una nueva entrada desconectada, con el contador de mensajes a cero. `UNREGISTER` busca el usuario, elimina su entrada y libera todos sus mensajes pendientes. `CONNECT` valida que el usuario exista y no esté ya conectado, guarda la IP obtenida por `accept` y el puerto enviado por el cliente, y marca al usuario como conectado. Tras una conexión correcta, intenta entregar todos los mensajes pendientes de ese usuario. `DISCONNECT` borra IP y puerto y marca el usuario como desconectado.

`USERS` comprueba que el solicitante exista y esté conectado. Si es así, calcula la lista de usuarios conectados y envía primero el número de entradas y después una cadena por usuario con el formato `usuario :: IP :: puerto`. Esta información es necesaria para que los clientes puedan contactar directamente entre sí para transferir ficheros.

El envío de mensajes se centraliza en una función común para `SEND` y `SENDATTACH`. El servidor valida que remitente y destinatario existan, incrementa el identificador del remitente y almacena siempre el mensaje en la cola del destinatario. Si el destinatario está conectado, intenta entregarlo inmediatamente mediante una conexión al puerto de escucha del cliente. Si la entrega tiene éxito, elimina el mensaje de la cola y envía una confirmación al remitente si este sigue conectado. Si la entrega falla, conserva el mensaje pendiente y marca al destinatario como desconectado.

Para `SENDATTACH`, el servidor utiliza las operaciones `SEND MESSAGE ATTACH` y `SEND MESS ATTACH ACK`. La diferencia respecto a `SEND` es que se conserva y propaga el nombre del fichero. El contenido del fichero no se almacena ni atraviesa el servidor principal.

El servidor integra el registro RPC llamando a `log_rpc_operation` cuando recibe operaciones de usuario. La llamada usa la variable de entorno `LOG_RPC_IP` para localizar el servidor RPC. Si la variable no está definida o la llamada falla, la operación de mensajería continúa, ya que el registro es auxiliar y no debe comprometer la funcionalidad principal.

1.7 7. Servicio web

El servicio web se implementa en `web_service.py` usando la biblioteca estándar de Python. Se despliega localmente en la máquina donde se ejecuta el cliente. Por defecto escucha en `127.0.0.1:8080`, aunque el puerto puede indicarse con la opción `-p`.

El objetivo del servicio es normalizar los mensajes antes de que entren en el sistema de mensajería. Recibe peticiones POST en la ruta `/normalize` con un cuerpo JSON que contiene el campo `message`. El servicio transforma el texto aplicando una separación por espacios y reconstruyendo la cadena con un único espacio entre palabras. La respuesta es otro JSON con el mensaje normalizado.

El cliente obtiene la URL del servicio mediante la variable de entorno `WEB_SERVICE_URL`; si no está definida utiliza `http://127.0.0.1:8080/normalize`. Si la llamada al servicio web falla, el cliente no envía el mensaje y muestra un fallo de `SEND` o `SENDATTACH`. Esta decisión evita enviar mensajes sin pasar por la normalización exigida.

1.8 8. Servicio RPC

El servicio RPC tiene como objetivo registrar las operaciones que llegan al servidor de mensajería. Se utiliza `ONC-RPC` en C. La interfaz se define en `log_rpc.x` mediante una estructura `log_request` con tres cadenas: `user`, `operation` y `filename`. El campo `filename` se utiliza únicamente para `SENDATTACH`; en el resto de operaciones se envía vacío.

La interfaz contiene el programa `LOG_RPC_PROG`, versión `LOG_RPC_VERS`, y el procedimiento `LOG_OPERATION`. A partir del fichero `.x` se generan `log_rpc.h`, `log_rpc_clnt.c`, `log_rpc_svc.c` y `log_rpc_xdr.c` mediante `rpcgen`.

El servidor RPC se completa en `log_rpc_server.c`. La función de servicio imprime una línea por operación recibida. El formato de salida es:

- `Nombre_usuario OPERACION`
- `Nombre_usuario SENDATTACH /tmp/file.txt`

El servidor de mensajería actúa como cliente RPC mediante `log_rpc_clnt.c`. La función auxiliar lee `LOG_RPC_IP`, crea un cliente RPC UDP, configura un timeout corto y llama al procedimiento remoto. Si no se ha definido la variable de entorno, devuelve sin realizar llamada. Si el servidor RPC no está disponible, el fallo no impide que el servidor principal siga atendiendo la operación de mensajería.

1.9 9. Compilación y ejecución

Para limpiar ejecutables generados:

```
make clean
```

Para compilar el servidor principal y el servidor RPC:

```
make
```

El Makefile compila el servidor de mensajería enlazando los ficheros del cliente RPC y los ficheros generados por rpcgen. También compila el ejecutable log_rpc_server. Se utilizan las opciones -Wall, -Wextra, -pthread, la ruta de cabeceras de tirpc y el enlace con -ltirpc.

Primero se arranca el servicio web en cada máquina donde vaya a ejecutarse un cliente:

```
python3 web_service.py -p 8080
```

Si se usa otro puerto:

```
export WEB_SERVICE_URL=http://127.0.0.1:<puerto>/normalize
```

Después se arranca el servidor RPC:

```
./log_rpc_server
```

Antes de lanzar el servidor principal, se indica dónde está el servidor RPC:

```
export LOG_RPC_IP=127.0.0.1
```

```
./server -p 8888
```

Finalmente se ejecutan uno o varios clientes:

```
python3 client.py -s 127.0.0.1 -p 8888
```

1.10 10. Pruebas realizadas

La siguiente tabla se deja como plantilla para completar con los resultados finales obtenidos en el entorno de entrega.

Prueba	Comandos ejecutados	Resultado esperado	Resultado obtenido
Registro correcto	REGISTER alice	El cliente muestra registro correcto y el servidor registra el usuario.	
Registro duplicado	REGISTER alice dos veces	La segunda operación indica usuario ya existente.	
Baja de usuario	UNREGISTER alice	El usuario se elimina y sus pendientes se liberan.	

Prueba	Comandos ejecutados	Resultado esperado	Resultado obtenido
Conexión correcta	CONNECT alice	Se crea hilo de escucha y el usuario queda conectado.	
Conexión de usuario inexistente	CONNECT ghost	El cliente informa de que el usuario no existe.	
Desconexión	DISCONNECT alice	El usuario queda desconectado y se cierra el listener local.	
Listado de usuarios	USERS	Se muestran entradas usuario :: IP :: puerto.	
Envío normal	SEND bob hola mundo	El destinatario recibe el mensaje y el remitente recibe ACK.	
Mensaje pendiente	SEND bob mensaje con bob desconectado y después CONNECT bob	El mensaje se entrega al reconectar.	
Envío con adjunto	SENDATTACH bob mensaje /tmp/a.txt	Se entrega el mensaje con línea FILE /tmp/a.txt y ACK específico.	
Transferencia de fichero	GETFILE alice /tmp/a.txt /tmp/copia.txt	El fichero remoto se copia por sockets entre clientes.	
Normalización web	SEND bob hola mundo	El receptor ve hola mundo.	
Registro RPC	Operaciones REGISTER, CONNECT, SEND, SENDATTACH	El servidor RPC imprime una línea por operación.	
Servicio web no disponible	Parar web_service.py y ejecutar SEND	El cliente informa de fallo y no envía sin normalizar.	
RPC no disponible	Parar log_rpc_server y ejecutar operaciones	La mensajería sigue funcionando aunque no se registre por RPC.	

1.11 11. Problemas encontrados y soluciones

Uno de los aspectos más importantes fue respetar el protocolo basado en cadenas terminadas en \0. TCP no conserva fronteras de mensaje, por lo que no basta con asumir que cada `recv` contiene un campo completo.

Para resolverlo se implementaron funciones auxiliares que leen hasta encontrar el byte nulo y que limitan el tamaño máximo del buffer.

La concurrencia del servidor requirió proteger el estado global. La lista de usuarios y las colas de mensajes pendientes pueden ser accedidas por varios hilos simultáneamente. Se resolvió mediante un mutex global. A la vez, se evitó mantener el mutex durante conexiones de red a clientes, ya que una entrega lenta podría bloquear el resto del servidor. La solución fue copiar los datos necesarios bajo bloqueo y realizar la comunicación fuera de la sección crítica.

Otro punto delicado fue la gestión de mensajes pendientes. El servidor debe almacenar el mensaje antes de intentar entregarlo y solo eliminarlo cuando la entrega se completa correctamente. Si falla la conexión con el destinatario, el mensaje se conserva y el usuario se marca como desconectado para reintentar la entrega más adelante.

La ampliación SENDATTACH podía haber roto SEND si se modificaba el protocolo existente. Para evitarlo se mantuvieron operaciones separadas: SEND MESSAGE y SEND MESS ACK para mensajes normales, y SEND MESSAGE ATTACH y SEND MESS ATTACH ACK para mensajes con fichero. La estructura de mensajes pendientes se amplió con un indicador de adjunto y un campo de nombre de fichero.

La transferencia de ficheros no podía implementarse con cadenas terminadas en \0, porque el contenido puede ser binario y contener bytes nulos. Por ello, en GETFILE se envió primero un estado, después el tamaño del fichero como cadena y finalmente el contenido como bytes exactos. Esto permite transferir tanto texto como datos binarios.

La integración RPC depende de servicios del sistema y de la disponibilidad del servidor RPC. Para que un fallo de registro no afecte a la mensajería, la llamada RPC se diseñó como auxiliar. Si LOG_RPC_IP no está definida o el servidor RPC no responde, la operación principal continúa.

1.12 12. Conclusiones

La práctica ha permitido implementar un sistema distribuido completo con varios mecanismos de comunicación. La parte principal usa sockets TCP y concurrencia con hilos para ofrecer un servicio de mensajería multiusuario. Sobre esta base se han incorporado mensajes con adjuntos, transferencia directa de ficheros entre clientes, normalización mediante servicio web y registro de operaciones mediante ONC-RPC.

La separación de responsabilidades facilita mantener el sistema. El servidor principal conserva el estado global de mensajería, los clientes gestionan la interacción con el usuario y las transferencias directas, el servicio web se limita a normalizar texto y el servidor RPC registra operaciones. Esta división permite ampliar la funcionalidad sin mezclar responsabilidades ni modificar innecesariamente el protocolo ya probado.

Los principales retos han estado relacionados con la concurrencia, el manejo correcto de cadenas terminadas en nulo, la entrega diferida de mensajes y la combinación de protocolos distintos. La solución final mantiene la funcionalidad obligatoria de la primera parte y añade las ampliaciones solicitadas de forma incremental.